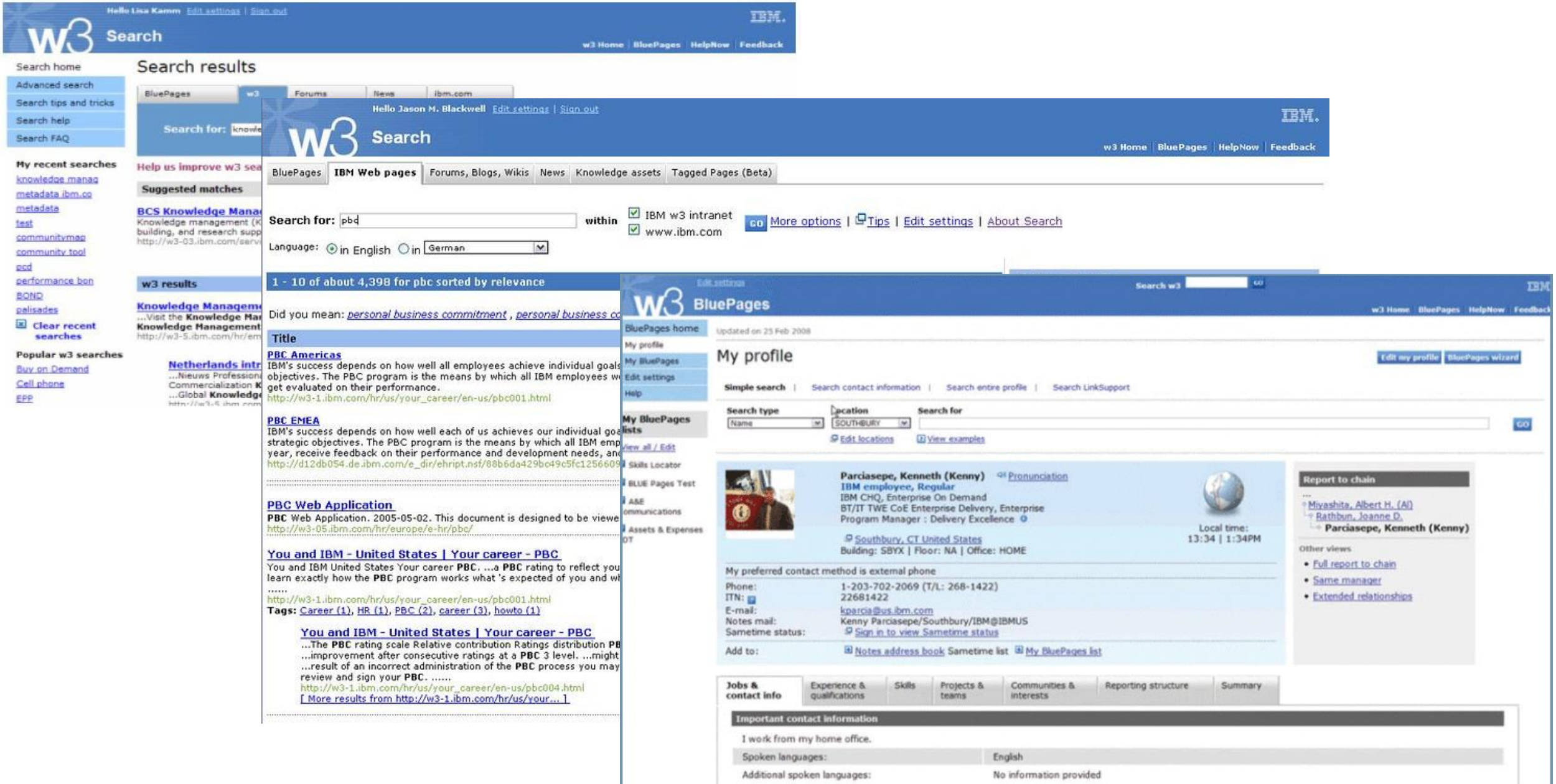
Az Search

https://github.com/iportilla/azure-ai-search-101

Fall 2026

https://github.com/Azure-Samples/azure-search-sample-data

IBM Internal Search (W3 Search)



Jen realized she'd lost Bruce when she found his iGlasses on a shelf in a used bookstore on Castro Street. They'd planned to meet up after her walk through Golden Gate Park. But now what? He could be anywhere. Deciding she'd rather spend a little money than a lot of time, Jen fired up Google's PeopleFinder.



https://learning.oreilly.com/library/view/search-patterns/9781449380205/ch06.html#searchvalence

Fortunately, Bruce wasn't sleeping in the park like last time. A camera picked him up near Frederick and Stanyan.

Google **PeopleFinder**

Bruce has been located near the corner of Frederick St. and Stanyan St.



Options

Save Video
Track by Camera
Contact (Crowdsource)

Jen approved the task budget, and Mechanical Turk found a worker nearby. Bruce agreed to meet back at the bookstore.



https://learning.oreilly.com/library/view/search-patterns/9781449380205/ch06.html#searchvalence

Debrief

- **Pervasive sensing:** Cameras, microphones, wearables, RFID, and face and voice recognition
- **Shared perspectives:** See and hear through others' eyes and ears, with permission
- **Life caching:** Pay to relive history as it happened
- **Search challenge:** Best insights may come from crowd behavior
- **Trade-off:** Spooky, but full of possibilities

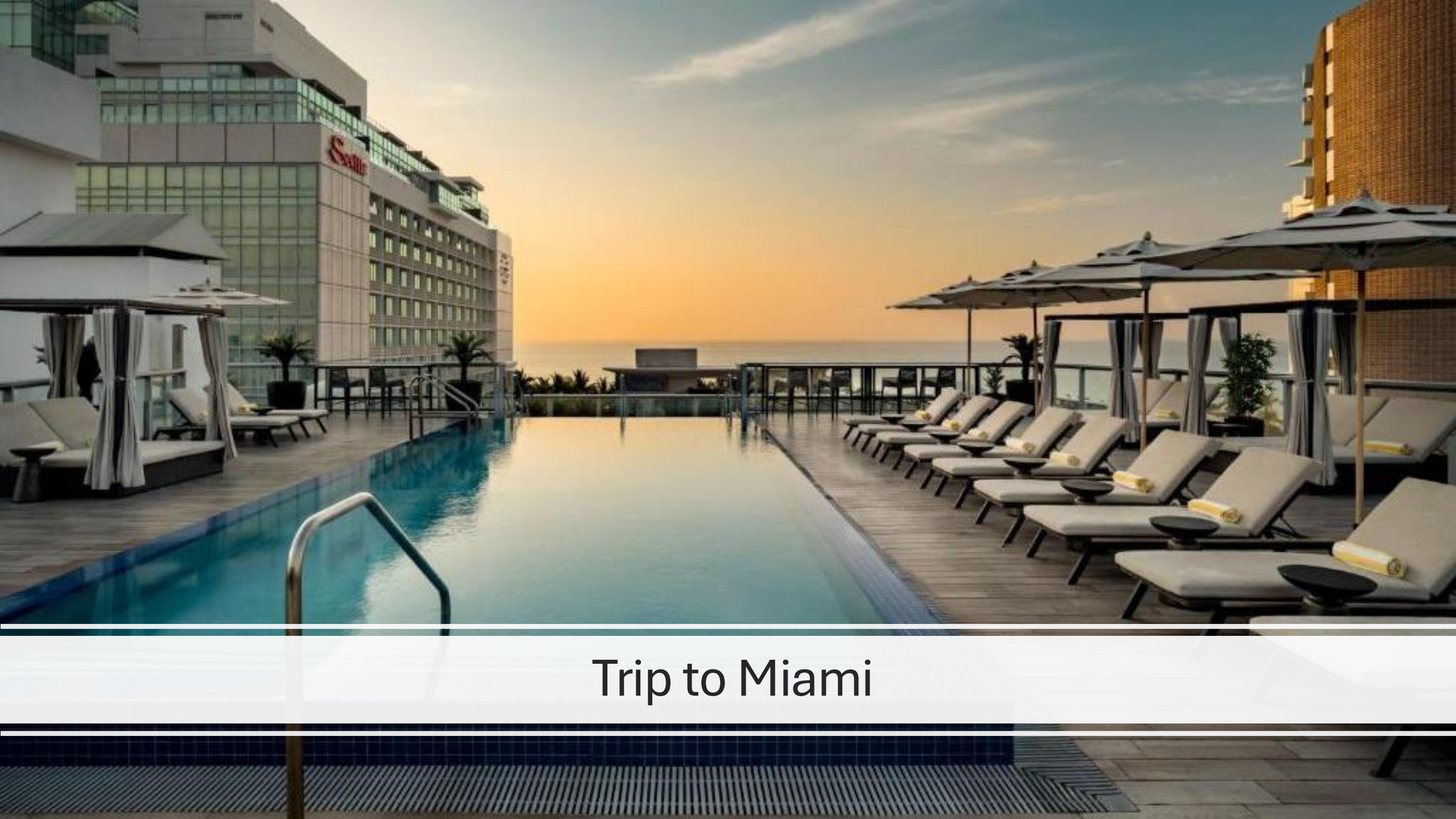
https://learning.oreilly.com/library/view/search-patterns/9781449380205/ch06.html#searchvalence

Debrief

In urban environments, we're already surrounded by video cameras, some integrated with microphone surveillance systems that can recognize a discrete sound, identify the point of origin, and turn cameras to focus precisely on that location. Imagine these cameras networked and their streaming video and audio available to the public for a fee. Now add iGlasses with Life Caching A/V worn by everyone who's anyone, and a layer of sophisticated face and voice recognition software. Maybe we'll have smart sidewalks and floors to track people by their unique footfall signatures. And don't forget the radio-frequency identification (RFID) tags in our wallets, shoes, and underwear—and perhaps a FriendChip™ or two under our skin.

Sure, this prospect may seem a bit spooky, but it's worth thinking through some of the intriguing possibilities. For instance, what if we could watch and listen through another's eyes and ears, assuming they gave us permission? This is already happening in a crude way. Just watch all the cellphones held high during an Olympic Opening Ceremony. And don't forget the life cache—we'll be able to see and hear history in the making, for a price. Designing search for this scenario should be interesting, especially since the best insights may emerge from the collective motion and behavior of crowds.

https://learning.oreilly.com/library/view/search-patterns/9781449380205/ch06.html#searchvalence



Trip to Miami

Search Queries

query = "wifi pool"

Category: Boutique

cozy place near touris



https://github.com/iportilla/azure-ai-search-101

Your turn

marriott.com

booking.com



airbnb.com

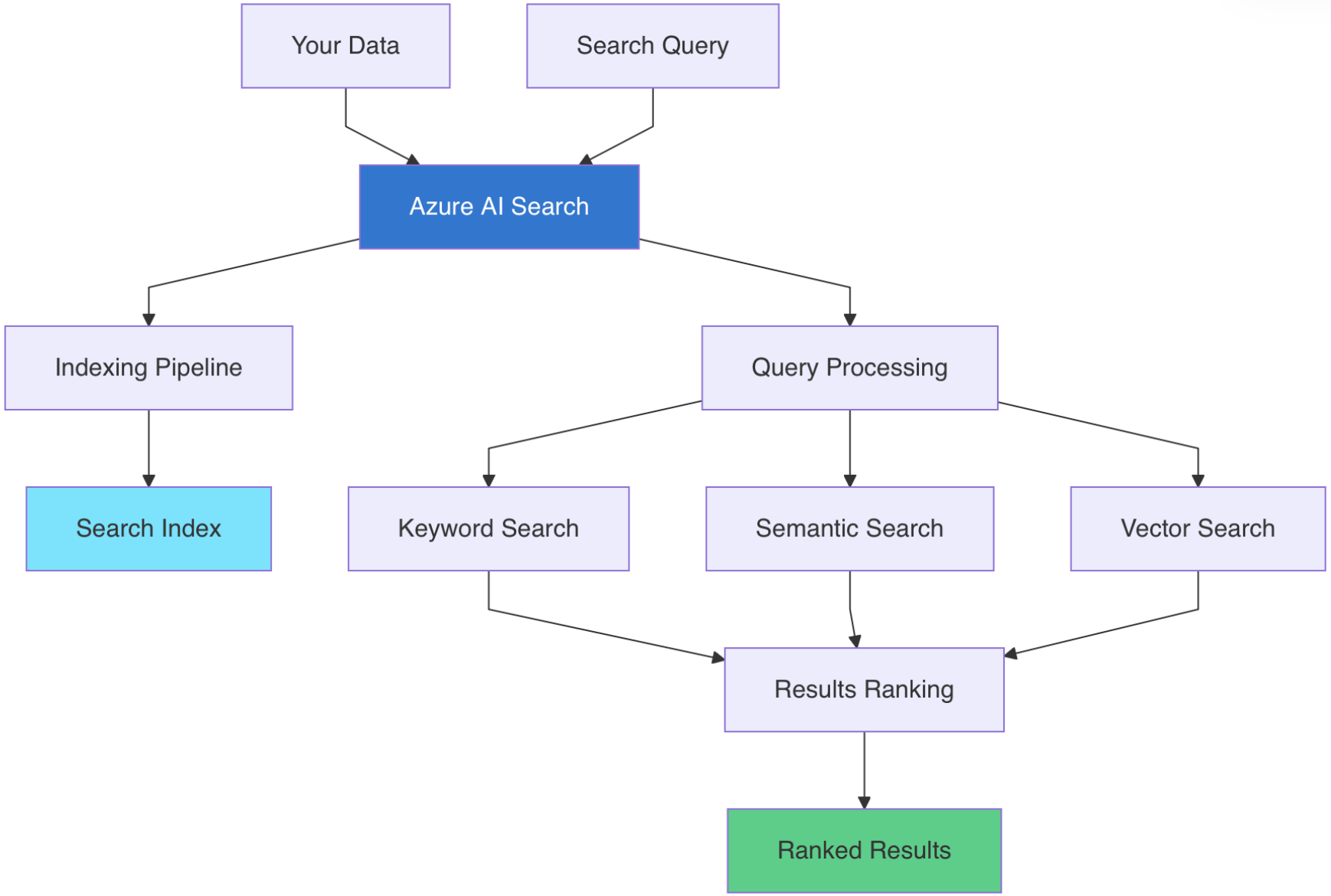
expedia.com

hilton.com

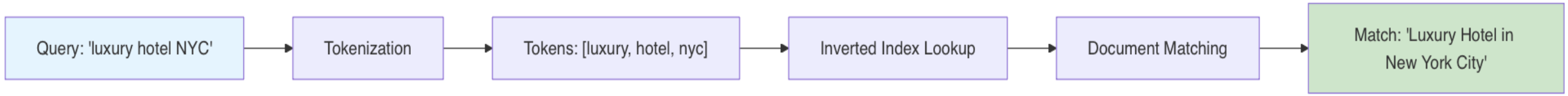
https://github.com/iportilla/azure-ai-search-101

Fields	Attributes
`HotelId`	Key, Retrievable, Filterable, Sortable, Searchable
`HotelName`, `Category`	Retrievable, Filterable, Sortable, Searchable
`Description`, `Description_fr`	Retrievable, Searchable
`Tags`	Retrievable, Filterable, Searchable
`ParkingIncluded`, `IsDeleted`	Retrievable, Filterable, Facetable
`LastRenovationDate`, `Rating`, `Location`	Retrievable, Filterable, Sortable
`Address.StreetAddress`, `Rooms.Description`, `Rooms.Description_fr`	Retrievable, Searchable
`Address.City`, `Address.StateProvince`, `Address.PostalCode`, `Address.Country`	Retrievable, Filterable, Facetable, Searchable, Sortable
`Rooms.Type`, `Rooms.BedOptions`, `Rooms.Tags`	Retrievable, Filterable, Facetable, Searchable
`Rooms.BaseRate`, `Rooms.SleepsCount`, `Rooms.SmokingAllowed`	Retrievable, Filterable, Facetable

Az AI Search


Your Data
Search Query
Azure AI Search
Indexing Pipeline
Query Processing
Search Index
Keyword Search
Semantic Search
Vector Search
Results Ranking
Ranked Results

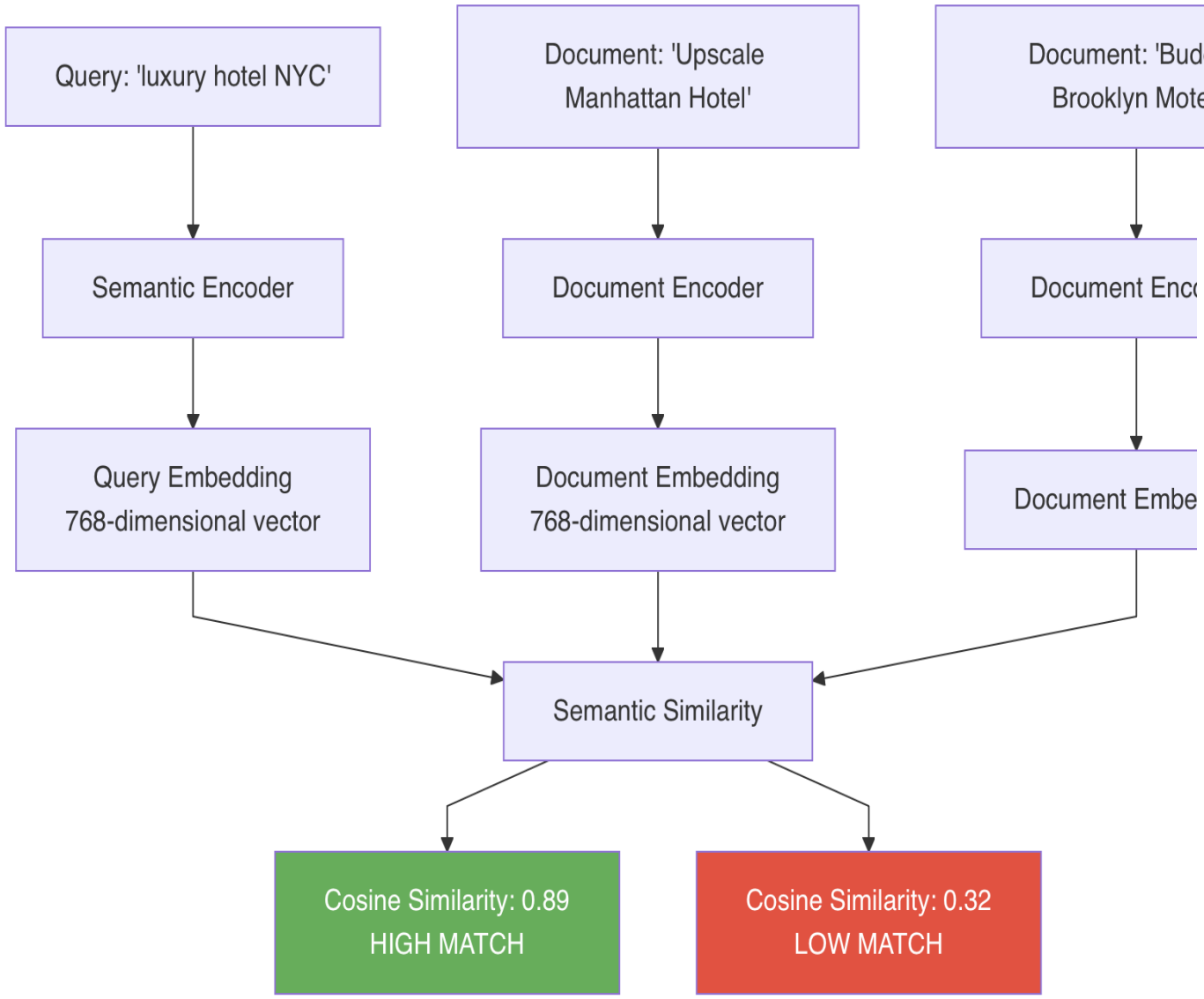
Keyword Search



Characteristics:

- ✅ Fast and efficient
- ✅ Exact word matching
- ✅ Works with analyzers (stemming, synonyms)
- ❌ No understanding of meaning
- ❌ Misses semantic variations
- ❌ Order matters for proximity

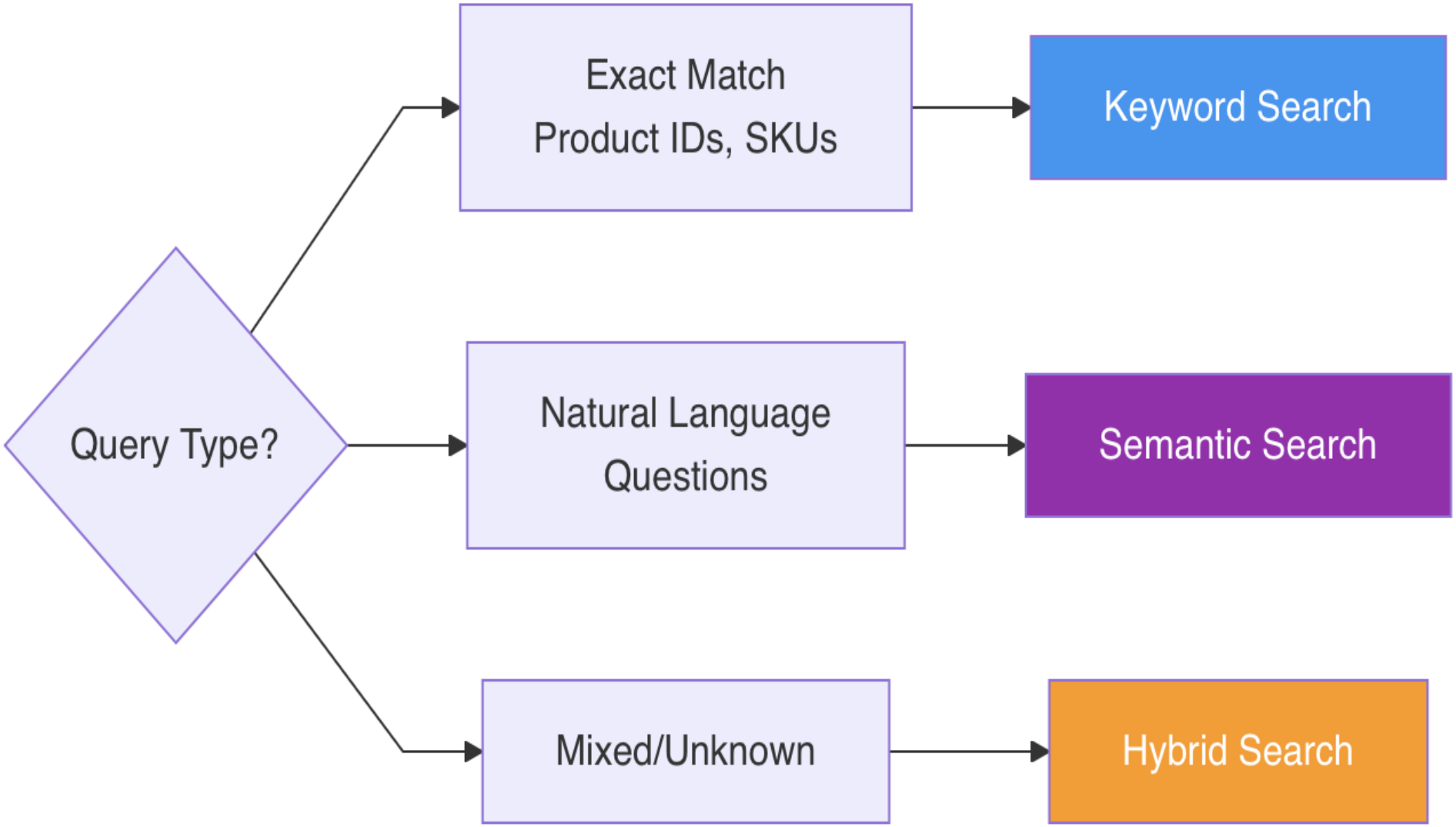
Semantic Search



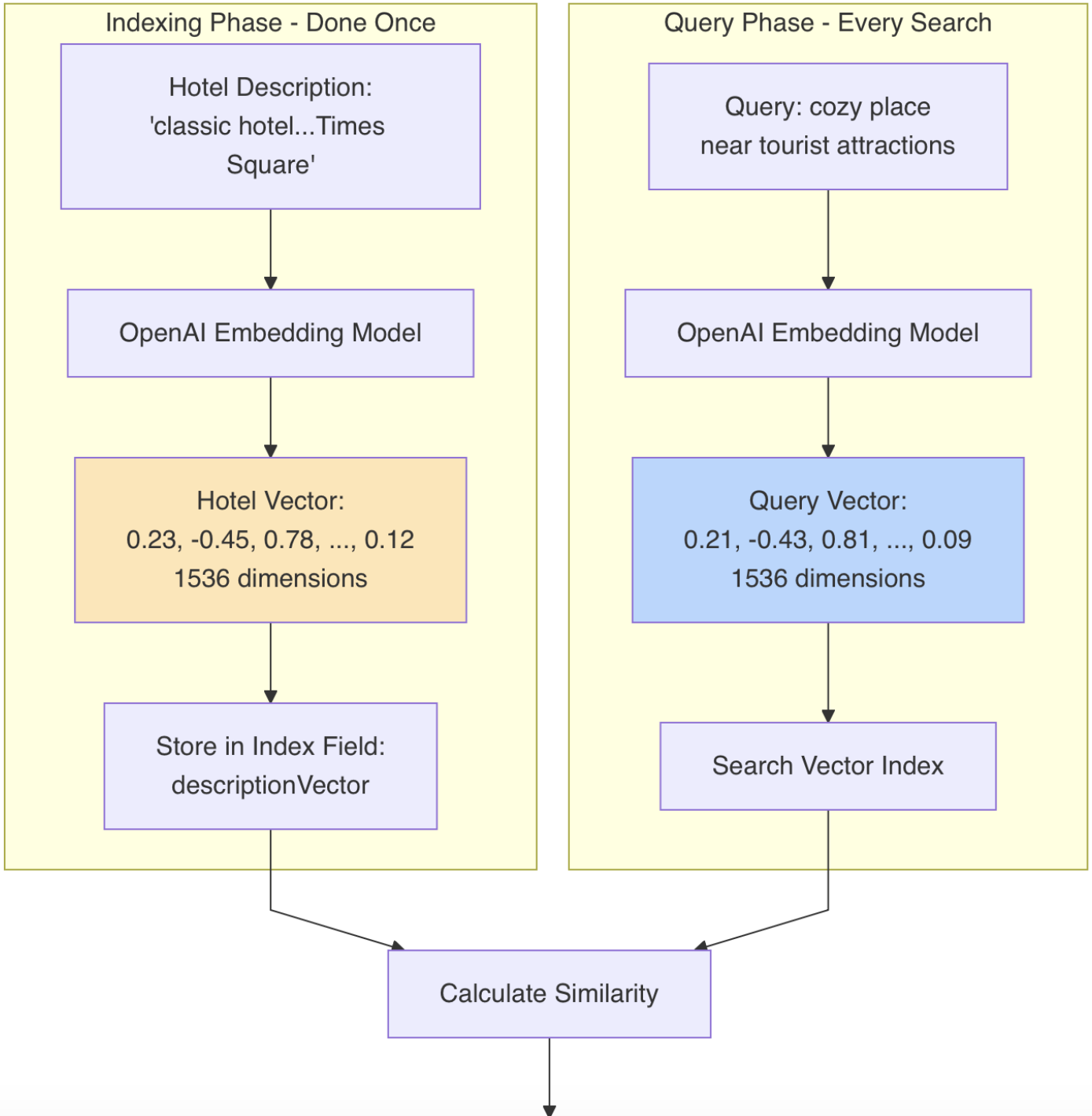
Characteristics:

- ✅ Understands meaning and context
- ✅ Matches semantic variations
- ✅ Works with natural language queries
- ✅ Handles synonyms automatically
- ❌ Slightly slower than keyword search
- ❌ Requires semantic configuration

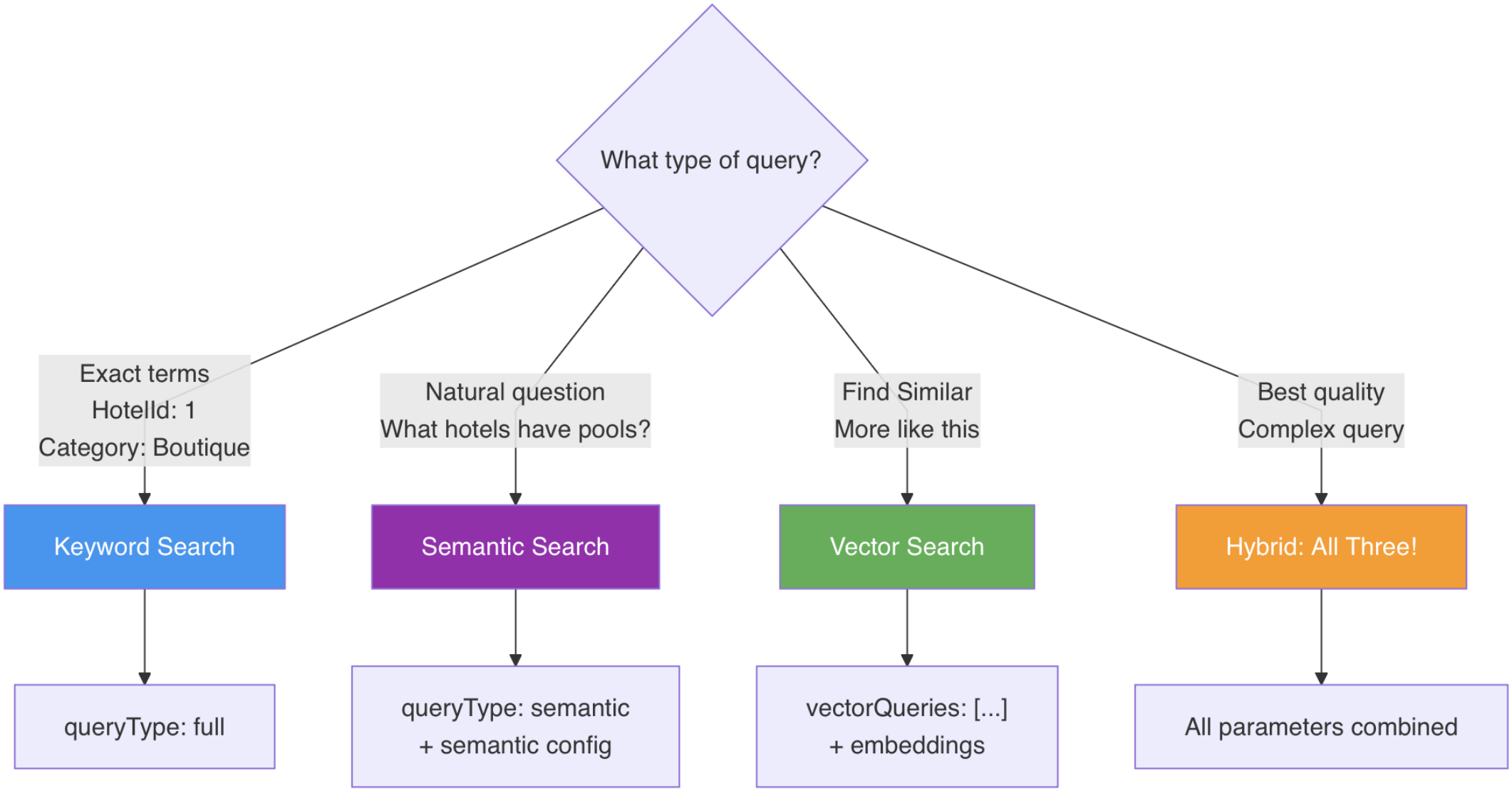
When to use what



Vector Search



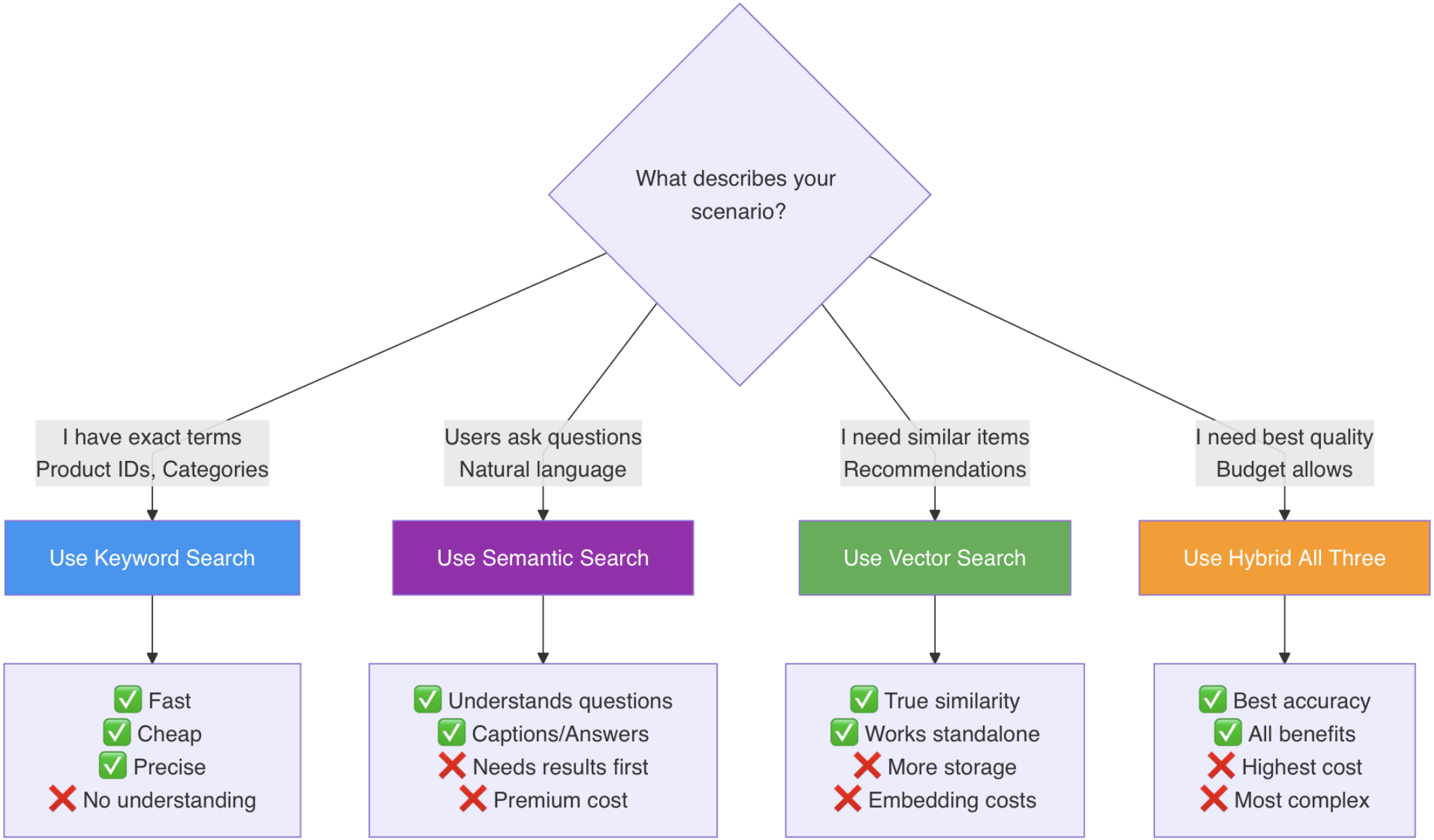
When to use each



Semantic vs Vector

Aspect	Semantic Search	Vector Search
What it is	Azure's built-in AI re-ranker	Custom similarity search using embeddings
Index storage	No vectors needed	Vectors stored in `Collection(Edm.Single)` fields
Setup	Configure semantic config	Create embeddings + vector fields
When it runs	After initial search (re-ranking)	Primary search method
Model	Microsoft's proprietary	Your choice (OpenAI, custom, etc.)
Input	Regular text fields	Pre-computed vector fields
Output	Re-ranked results + captions + answers	Similarity-ranked documents
Query type	`queryType: "semantic"`	`vectorQueries: [...]`
Cost	Premium tier + per-query	Storage for vectors + embedding costs

Summary


What describes your scenario?
I have exact terms
Product IDs, Categories
Users ask questions
Natural language
I need similar items
Recommendations
I need best quality
Budget allows
Use Keyword Search
Use Semantic Search
Use Vector Search
Use Hybrid All Three
✅ Fast
✅ Cheap
✅ Precise
❌ No understanding
✅ Understands questions
✅ Captions/Answers
❌ Needs results first
❌ Premium cost
✅ True similarity
✅ Works standalone
❌ More storage
❌ Embedding costs
✅ Best accuracy
✅ All benefits
❌ Highest cost
❌ Most complex